

Control de un LED mediante Bluetooth desde una aplicación móvil

David Cesar Nicanor y Garcia, Mónica Leonor Cuautle Nava, Andrade Barrales Luis, Cristian Geovany Cuautle Pichardo

Facultad de Ciencias de la Computación

Resumen—En este proyecto se diseñó una aplicación móvil capaz de controlar el encendido y apagado de un LED a través de la tecnología Bluetooth utilizando un módulo compatible (como el HC-05 o BT04-A). El propósito principal fue demostrar la comunicación entre un dispositivo móvil (Android) y un microcontrolador (Arduino) mediante una interfaz sencilla e intuitiva. Adicionalmente, se implementaron funciones de parpadeo (BLINK) y control de brillo mediante el uso de modulación por ancho de pulso (PWM), controlado desde un *Slider* en la aplicación. Los resultados obtenidos confirman el correcto funcionamiento del sistema, cumpliendo con los objetivos planteados de control de estado e intensidad.

Index Terms—Bluetooth (BT) Communication, Microcontroler (MCU), Arduino, Mobile Application, Android, Kotlin, Pulse Width Modulation (PWM), LED Control, Serial Port Profile (SPP), Internet of Things (IoT)

I. INTRODUCCIÓN

El desarrollo de las tecnologías de comunicación inalámbrica ha permitido integrar dispositivos móviles con microcontroladores para la manipulación remota de periféricos electrónicos. En este proyecto, se utilizó la tecnología Bluetooth como medio de transmisión para encender, apagar, hacer parpadear y controlar la **intensidad (brillo)** de un LED desde una aplicación móvil desarrollada en Android Studio con Kotlin. Esta práctica tiene como finalidad comprender el funcionamiento del protocolo Bluetooth (SPP) [1], la programación del módulo de comunicación y la lógica de control implementada en el microcontrolador, incluyendo el manejo de PWM [2] y temporizadores no bloqueantes.

II. OBJETIVOS

- Diseñar una aplicación móvil capaz de manipular el encendido y apagado de un LED a través de la tecnología Bluetooth con ayuda de un módulo compatible (HC-05, BT04-A, etc.). (Puntaje máximo aspirado: 75 %)
- Agregar una función o botón de **parpadeo** (BLINK), que se ejecute en el microcontrolador de forma no bloqueante. (Puntaje máximo aspirado: 80–90 %)
- Implementar mediante el uso del PWM un control de **brillo variable** para el LED, controlado desde un *Slider* (barra deslizante) en la aplicación móvil. (Puntaje máximo aspirado: 100 %)

III. DESARROLLO DEL PROYECTO

El desarrollo se dividió en dos partes principales: la creación de la aplicación móvil y la programación del microcontrolador. La aplicación fue diseñada en **Android Studio** utilizando **Kotlin** [3], [4] y el patrón *ViewBinding*. Se agregaron botones para las acciones de “Encender”, “Apagar” y “Parpadear”, además de un control deslizante (*Slider* o *SeekBar*) para ajustar el brillo del LED. En el microcontrolador (Arduino), se configuró la comunicación serial con el módulo Bluetooth (ej. BT04-A) [1] para interpretar los comandos enviados desde la aplicación (“ON”, “OFF”, “BLINK”, “PWM=[valor]”) y ejecutar las acciones correspondientes sobre el pin de salida (Pin 9, con capacidad PWM).

IV. FIRMWARE DE ARDUINO

El firmware fue desarrollado utilizando el entorno de programación Arduino IDE. En el código se estableció la comunicación serial a 9600 baudios, tanto para el monitor serial USB como para el módulo Bluetooth (conectado vía *SoftwareSerial* a los pines 10 y 11) [1]. El pin 9, con capacidad PWM, se configuró como salida para el LED.

La lógica principal reside en el `loop()`. Primero, comprueba si hay datos disponibles en el puerto serial del Bluetooth (`BTSerial.available()`). Si es así, lee el comando entrante como un `String` hasta recibir el carácter de nueva línea `\n`. El comando se limpia de espacios en blanco y se procesa:

- Si el comando es “ON”, la máquina de estados (`ledState`) cambia a 1 (Sólido) y el brillo (`ledBrightness`) se fija en 255.
- Si es “OFF”, `ledState` cambia a 0 (Apagado).
- Si es “BLINK”, `ledState` cambia a 2 (Parpadeo).
- Si el comando inicia con “PWM=”, el estado cambia a 1 (Sólido), se extrae el valor numérico después del “=”, y se asigna a `ledBrightness`, asegurando que esté entre 0 y 255.

Inmediatamente después de procesar un posible comando, un bloque `switch(ledState)` ejecuta la acción correspondiente al estado actual. En el estado 2 (BLINK), se utiliza un temporizador no bloqueante basado en `millis()` que invierte el estado del LED cada 500 milisegundos sin detener el resto del programa. En los estados 0 (OFF) y 1 (ON), se utiliza `analogWrite()` para apagar el LED (valor 0) o encenderlo con la intensidad almacenada en `ledBrightness` [2].

V. APLICACIÓN MÓVIL

La aplicación fue desarrollada en **Android Studio** utilizando el lenguaje de programación **Kotlin** [3], [4]. Se usó *ViewBinding* para interactuar con los elementos de la interfaz.

La interfaz, cuya estructura se muestra en la Figura 1, contiene:

- **Botón “Encender LED”**: Envía el comando “ON\n”.
- **Botón “Apagar LED”**: Envía el comando “OFF\n”.
- **Botón “Parpadear”**: Envía el comando “BLINK\n”.
- **SeekBar “Luminosidad”**: Envía el comando “PWM=[valor]\n” solo al soltar el control, para evitar la saturación del canal Bluetooth.

Se utiliza el protocolo **SPP (Serial Port Profile)** [1] con el **UUID** estándar 00001101-0000-1000-8000-00805F9B34FB. La aplicación gestiona los permisos de Bluetooth modernos (para Android 12+) y solicita al usuario activar el Bluetooth si está deshabilitado.

La conexión se realiza de forma asíncrona utilizando **Corutinas de Kotlin** (`lifecycleScope.launch(Dispatchers.IO)`) para no bloquear el hilo principal (UI). Se busca el dispositivo emparejado con el nombre BT04-A y se establece la conexión. El envío de datos se realiza a través del *socket* RFCOMM.

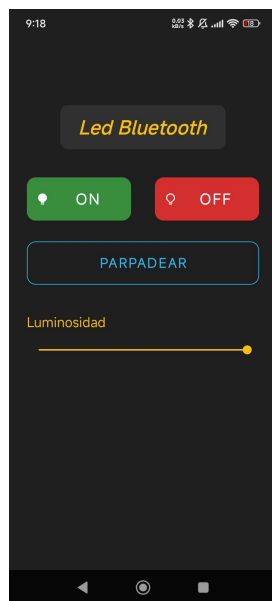


Figura 1. Interfaz gráfica de la aplicación móvil desarrollada.

La lógica de la interfaz se concentra en la función `setupListeners()`. Los botones “ON”, “OFF” y “BLINK” simplemente llaman a la función `sendCommand()` con sus respectivos comandos. Los botones “ON” y “OFF” también actualizan la posición del *Slider* a 255 y 0 respectivamente, para mantener la coherencia visual.

El componente clave es el *SeekBar*, que utiliza el *listener* `onStopTrackingTouch()`. Esto es crucial, ya que el

comando “PWM=[valor]” solo se envía cuando el usuario levanta el dedo del control deslizante. Esta técnica evita la saturación del búfer serial del Bluetooth, que ocurriría si se enviaran comandos en tiempo real mientras se arrastra el control.

La función `sendCommand(command: String)` verifica si el *socket* está conectado. Si lo está, lanza una corutina en el hilo de `Dispatchers.IO` (optimizado para operaciones de entrada/salida) para enviar los datos. Añade el carácter `\n` al comando y lo escribe en el `outputStream` del *socket*, dentro de un bloque `try-catch` para manejar posibles errores de conexión.

VI. RESULTADOS

El sistema logró establecer una conexión estable con el módulo Bluetooth BT04-A y respondió correctamente a las órdenes enviadas desde la aplicación. Las pruebas demostraron que la latencia fue imperceptible. Las funciones de encendido, apagado y parpadeo operaron según lo esperado. Adicionalmente, se verificó exitosamente el control de brillo (PWM) [2]. La aplicación pudo enviar valores desde 0 hasta 255 mediante el *Slider*, y el LED respondió cambiando su intensidad lumínica de forma proporcional y fluida, como se observa en las Figuras 2 y 3. El montaje final del circuito se presenta en la Figura 4.

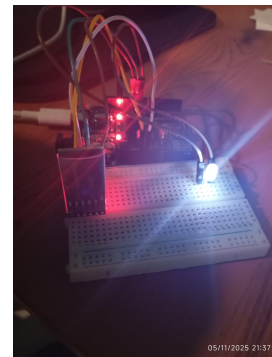


Figura 2. Prueba de PWM con luminosidad baja (ej. valor 50).

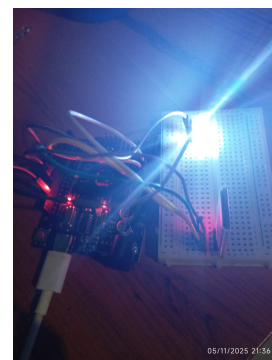


Figura 3. Prueba de PWM con luminosidad alta (valor 255).

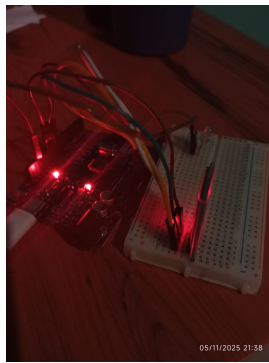


Figura 4. Resultado final.

VII. MEJORAS DE VERSIÓN

La versión inicial del proyecto se centraba en un sistema de control unidireccional (App → Arduino). La versión actualizada del firmware y del software expande el proyecto a un sistema **bidireccional e interactivo** (App ↔ Arduino), añadiendo retroalimentación basada en sensores.

Las mejoras principales se detallan a continuación.

VII-A. Firmware (Arduino)

Se incorporó un sensor de luz digital (módulo LM393) conectado al pin digital 8. El firmware ahora implementa una lógica de detección de ambiente:

- El microcontrolador monitorea constantemente el estado del sensor de luz.
- Se implementó un temporizador no bloqueante (basado en `millis()`) que se activa cuando se detecta oscuridad.

Fragmento funcionamiento

```
// Sensor de luz
int luz = digitalRead(sensorPin);
unsigned long currentMillis_sensor=millis();

if (luz == HIGH) { // HIGH = oscuridad
  if (darkTimer == 0) {
    // Si es la primera vez que se detecta
    //oscuridad se inicia el timer
    darkTimer = currentMillis_sensor;
  }

  // si ha estado oscuro por MAS de 3 segundos
  if(currentMillis_sensor-darkTimer>darkThreshold){
    // Y si no se ha enviado la notificacion todavia
    if (!darkNotificationSent) {
      BTSerial.println("DARK");
      Serial.println("Enviando 'DARK' al movil");
      darkNotificationSent = true;
    }
  }
} else { // significa que hay LUZ (luz == LOW)
  if (darkTimer > 0) {
    Serial.println("Luz detectada, reseteando timer.");
  }
  darkTimer = 0; // Reset del temporizador
  darkNotificationSent = false;
}
```

- Si la condición de oscuridad persiste por un tiempo predefinido (`darkThreshold` de 3 segundos), el Arduino transiciona de ser un receptor a ser un emisor.

- Se envía activamente el comando de texto “DARK\n” a través del `BTSerial` hacia la aplicación móvil conectada. La lógica de detección sin el LED encendido se muestra en la Figura 5.
- Se utiliza una bandera (`darkNotificationSent`) para asegurar que este mensaje se envíe una sola vez por evento de oscuridad, evitando la saturación del canal.

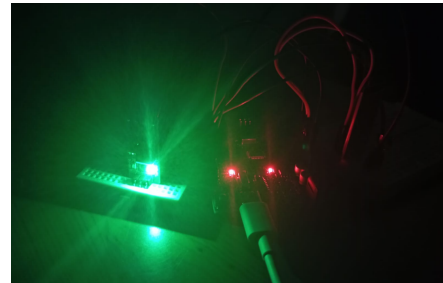


Figura 5. Estado al detectar oscuridad sin encender el LED.

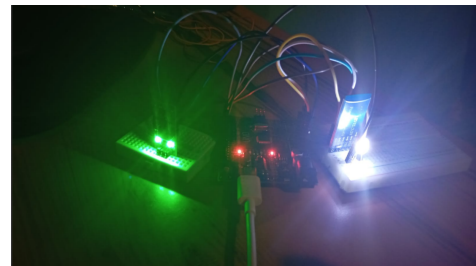


Figura 6. Estado al detectar oscuridad con el LED encendido.

VII-B. Aplicación móvil (Kotlin)

La aplicación móvil fue actualizada significativamente para manejar la nueva comunicación bidireccional:

- **Escucha Asíncrona:** Se implementó una corutina (`lifecycleScope.launch(Dispatchers.IO)`) que ejecuta la función `startListeningToBluetooth()`. Esta función monitorea de forma constante el `InputStream` del socket Bluetooth, permitiendo a la app recibir datos del Arduino.
- **Permisos de Notificación:** La app ahora solicita el permiso `POST_NOTIFICATIONS`, requerido por versiones modernas de Android (API 33+) para mostrar alertas.
- **Sistema de Notificación:** Al recibir el string “DARK” del Arduino, la app genera una notificación *push* local (ver Figura 7).
- **Notificaciones Interactivas:** La notificación no es solo informativa; incluye un botón de acción (“Encender”). Al presionarlo, la app gestiona el envío del comando “ON” al Arduino. La notificación *push* y el botón de acción se aprecian en la Figura 8.
- **Manejo de Acciones Pendientes:** La lógica `handleIntentAction` y `onNewIntent` permite

que la acción de la notificación funcione incluso si la app estaba cerrada. La app almacena la acción (ej. "ACTION_ON"), se conecta al Bluetooth y luego ejecuta el comando pendiente.

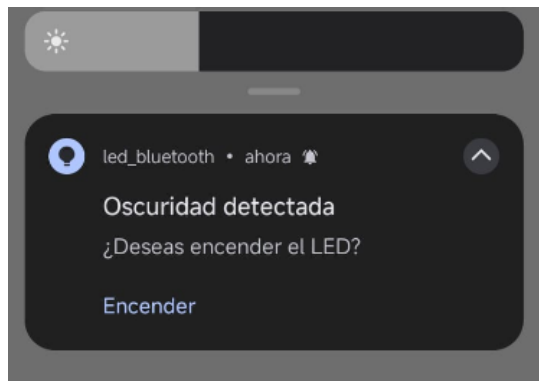


Figura 7. Notificación push local generada al detectar oscuridad.

y la gestión de permisos de notificación modernos (API 33+) permitió la creación de **alertas push interactivas** (ver Figura 7). Estas notificaciones con acciones pendientes integran la interfaz móvil funcional y amigable en Kotlin [4] con el control del sistema.

Finalmente, se cumplieron y superaron los objetivos establecidos, al demostrar la integración efectiva y robusta entre hardware (Arduino) y software (Android Studio), implementando una lógica de control moderna y eficiente que incluye el uso de la modulación PWM [2] para generar diferentes **intensidades de luz** en un LED.

REFERENCIAS

- [1] Módulo Bluetooth HC-05, "Datasheet and User Manual," JNHuaMao Technology Co., Ltd., 2016.
- [2] Texas Instruments, "Pulse Width Modulation (PWM) Techniques," Application Report, 2018.
- [3] Google, "Documentación Oficial de Android," *Android Developers*. [En línea]. Disponible: <https://developer.android.com>
- [4] Google, "Desarrollo de Android con Kotlin," *Android Developers*. [En línea]. Disponible: <https://developer.android.com/kotlin>

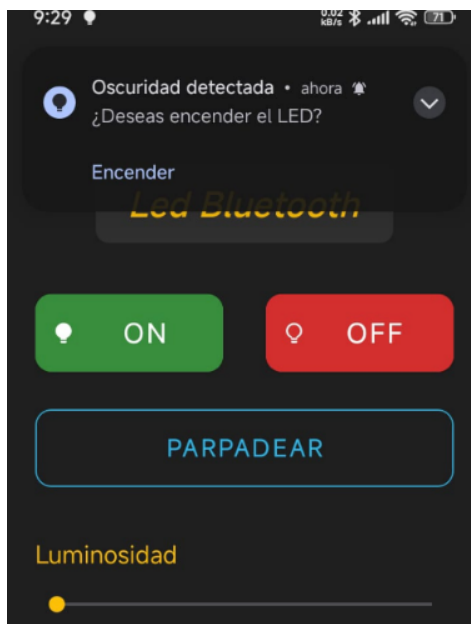


Figura 8. Interfaz gráfica mostrando la notificación con botón de acción.

VIII. CONCLUSIONES

El proyecto permitió comprender y aplicar con éxito los principios de la comunicación **bidireccional e interactiva** entre dispositivos Bluetooth (Android) [1], [3] y microcontroladores (Arduino).

La implementación de un **sensor de luz digital** amplió la funcionalidad del sistema, demostrando cómo el hardware puede proveer **retroalimentación ambiental** al software (Firmware). Esto culminó en el desarrollo de un mecanismo de detección de oscuridad con temporización no bloqueante.

Por el lado de la aplicación móvil, el uso de corutinas (`lifecycleScope.launch`) para la **escucha asíncrona**